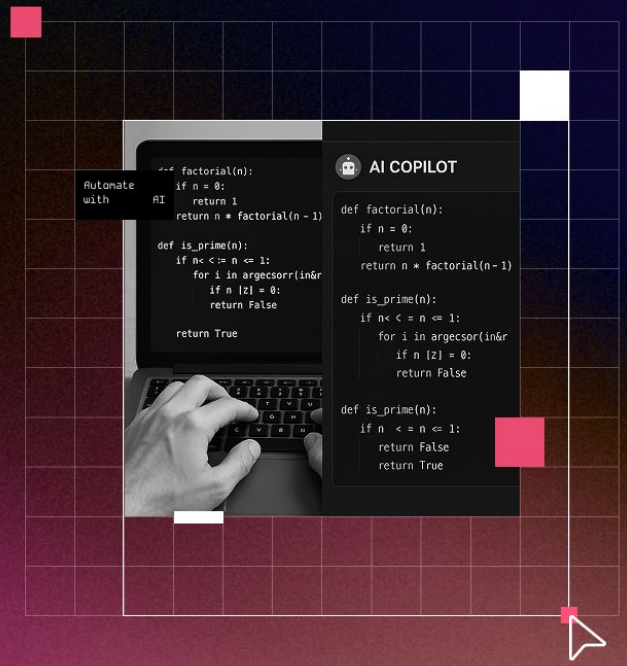
Airtribe

Agents in LangChain

When LLMs decide what to do next



Divij Bajaj

Sr. Data & Applied Scientist @Microsoft

Meet your Instructor

Divij Bajaj is a Data & Applied Scientist II at Microsoft with ~6 years of experience in building, productionizing, and scaling large-scale machine learning models for real-world business problems. He has worked extensively on end-to-end ML systems, from problem framing and data modeling to deployment and monitoring in production. Divij has delivered high-impact solutions across enterprise use cases and has been recognized for excellence through multiple competitive and academic achievements.

Divij Bajaj

Sr. Data and Applied Scientist @

Airtribe

Quick Recap from Last Session

- **LLMs are stateless** — every call starts fresh, no built-in memory
- **RAG pipeline** — query → embed → retrieve → generate using vector stores
- **LangChain as glue** — connects prompts, data, and logic into reusable chains
- **Memory bridges the gap** — injects past context via `{history}` into prompts
- **5 memory types** — Buffer, Window, Summary, SummaryBuffer, and Entity — each with different token/recall trade-offs

What We'll Do Today

- What an agent is
- How tools and the agent loop work
- How to build agents with memory
- How to avoid common traps
- How to debug agent behavior

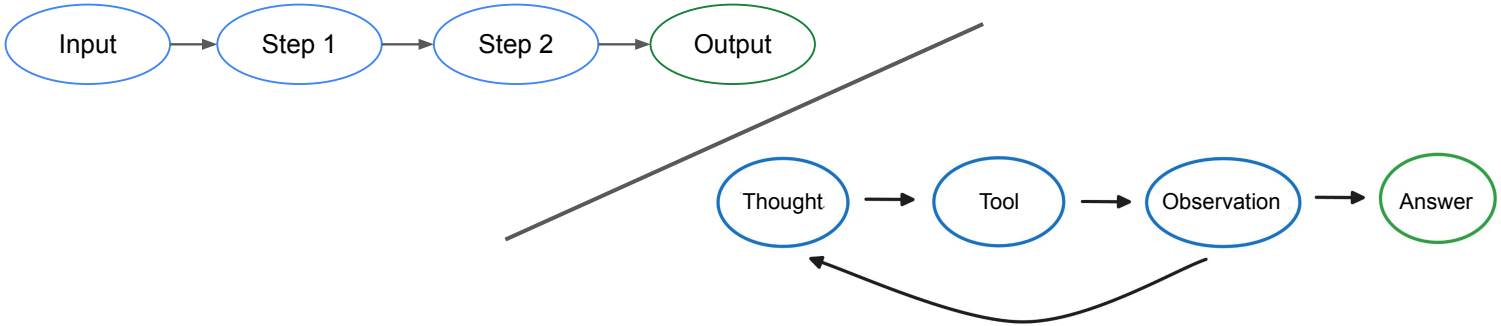
When to Use Agents (Learning Outcomes)

Learners will learn:

- How LLMs, LangChain, and RAG work together in AI applications.
- The difference between chains and agents.
- When it makes sense to use an agent and when it does not.
- How an agent thinks and decides what to do next.

Chains vs Agents

Chains	Agents
Follow a **fixed flow**	Model **selects the tool**
You define the **sequence**	Can **plan** → **act** → **observe** → **repeat**
Useful when steps are **predictable**	Useful when the path is **unclear at the start**



When to Use Agents

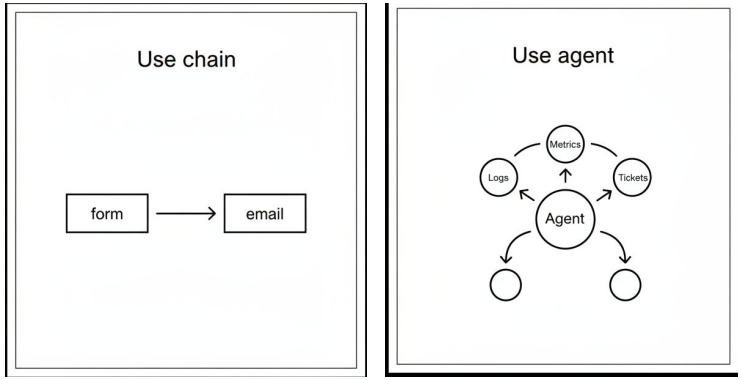
Airtribe

Use an agent when:

- System must **choose actions** based on context
- Tasks involve **research, troubleshooting, or planning**
- Path is **not obvious** before you start

Avoid agents when:

- Flow is **simple and fixed**
- You need **predictable latency and cost**
- Output format must be **strict and repeatable**

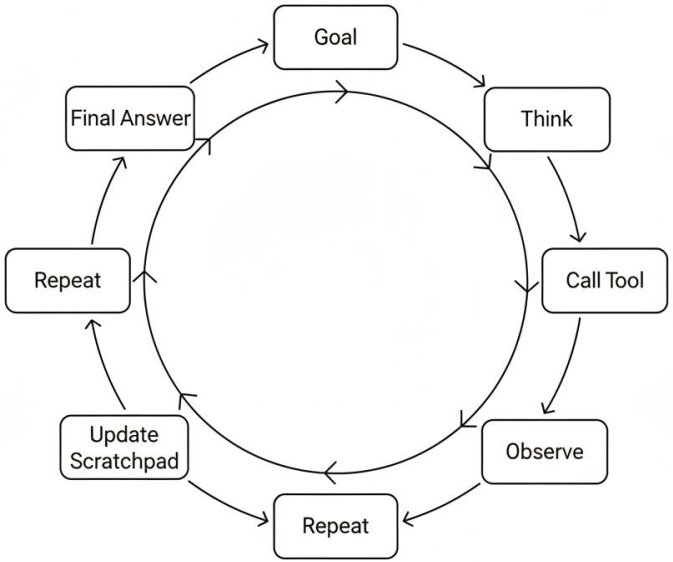


How an Agent Thinks (Mental Model)

Given a user goal, an agent:

1. Reads the **goal + scratchpad**
2. Decides if a **tool** is needed
3. Chooses a tool and **builds arguments**
4. Executes the tool
5. Appends the **result** to the scratchpad
6. Repeats until it can answer

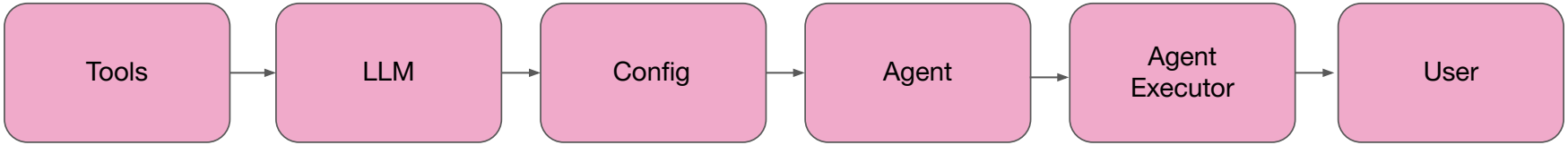
LangChain provides this loop so you don't write your own while True control logic.



Core Pieces of an Agent in LangChain

Airtribe

- **LLM / Chat model** – the brain
- **Tools** – functions & APIs it can call
- **Agent type**
  - Function calling agent
  - ReAct agent
  - Structured chat agent
- **AgentExecutor**
  - Connects all pieces
  - Runs the control loop



Tools, Memory & Guardrails (Learning Outcomes)

Learners will learn:

- What tools are and how to design them effectively.
- Why clear tool descriptions and schemas are important.
- How memory helps agents maintain context across conversations.

What Are Tools?

A **tool** contains:

- A **function**
- A **name**
- A **description**
- An **input schema**

Why it matters:

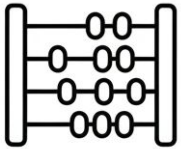
- Model sees only **names + descriptions**
- It prepares arguments based on the **schema**
- Bad descriptions = wrong tool calls

```
name: "search_logs" description:
"Search application logs by
service name and time range"
args: { service_name, from, to }
```

Demo: Calculator & API Tool

Airtribe

- `calculator` → add, subtract, multiply, divide
- `get_weather` → takes `city`, returns temperature & condition



- add, subtract
- multiply, dvide

- city → temperature
- condition

Steps to Build Your First Agent

1. Import the **model**
2. Create **tools**
3. Choose **agent type**
4. Initialize **AgentExecutor**
5. Run sample queries
6. Turn on **verbose** logs

Tool Design Principles

For each tool, define:	Avoid:
Action-style name: `search_logs`, `create_ticket`, `query_metrics`	Tools that **mix multiple jobs**
Clear arguments: `service_name`, `time_range`, `severity`	Outputs with **unpredictable structure**
Simple return format: text or basic JSON	

Agents with Memory and Guardrails

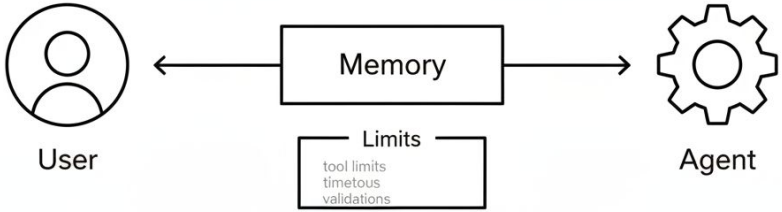
Airtribe

Why memory helps

- Keeps context across multiple turns
- Supports longer tasks and follow-ups

Ways to add memory

- Store **recent turns** (chat history)
- Use **summary memory** for long conversations



Controls you can add

- Limit number of **tool calls**
- Timeouts per run
- Restricted **tool sets** per user or task
- **Validate** inputs/outputs and use fallbacks

How to Debug Agent Behavior

- Enable **verbose logs**
- Log each **tool call** (name, input, output)
- Review **thought traces** (for ReAct)
- Reproduce issues with **small prompts**
- Fix by:
  - Improving **tool descriptions**
  - Adjusting **system prompt**
  - Reducing allowed **iterations/tools**

Common Agent Traps

- Too many tools → model gets confused
- Tools that overlap in meaning
- No limits → long, costly loops
- Complex outputs with no clear schema
- Using agents for simple, static workflows

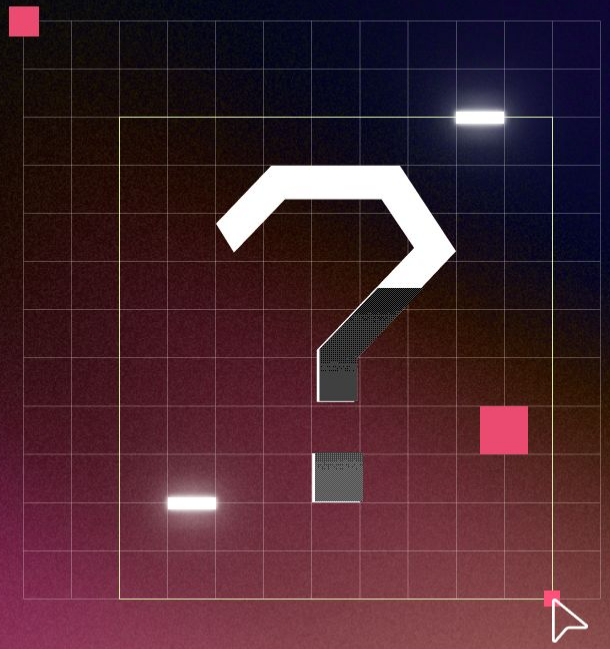
- Too many tools → model gets confused
- Tools that overlap in meaning
- No limits → long, costly loops
- Complex outputs with no clear schema
- Using agents for simple, static workflows

Best Practices for Using Agents

- Keep **actions small and clear**
- Use as **few tools as possible**
- Limit agent **freedom & loops**
- Test tool descriptions carefully
- **Log everything** in dev
- Start with **offline test prompts** before going live

What You Learned Today

- Difference between **chains and agents**
- When an **agent** makes sense
- How tools + agent loop work
- How to **add memory & guardrails**
- How to **debug and improve** agent behavior

Airtribe

QNA